PROGRAMMING · BEGINNER

# C++ Essentials

## Write clearer C++ with objects and ownership.

Free C++ foundations covering modern syntax, OOP, STL and practical problem solving.

6-8 weeks | 4 modules | 16 lessons

**Explore the course** →

Concepts, worked examples, assignments and a coherent course project.

WHO IT IS FOR · WHAT YOU WILL LEARN

# Your next capability.

Progress from C++ language fundamentals into classes, containers, algorithms, memory ownership and a portfolio-ready console application.

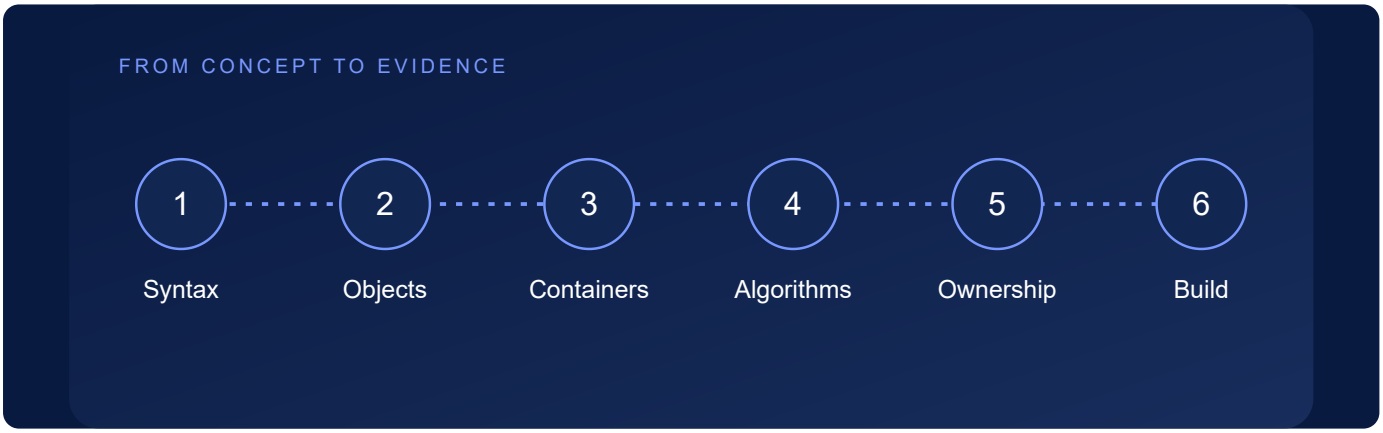
- Write modern C++ programs
- Apply object-oriented design
- Use STL containers and algorithms
- Understand memory and ownership basics

## Audience

- C learners progressing to C++
- Engineering students
- Aspiring systems developers

## Prerequisites

- Basic programming familiarity is helpful




### One curriculum, three connected views

This guide summarizes the same modules and lessons shown on the course overview and in the learning workspace. The next pages explain the scope and practical evidence for every module.

DETAILED MODULE CURRICULUM · 1–2

# Build the foundations.

The module names, lesson sequence and evidence below match the course learning workspace.

## 1 Modern C++ Foundations

Create a small modern C++ task utility with clear value semantics.

**1.1 Toolchain and first program**
Compile a minimal program and distinguish source, object files and the linked executable.

**1.2 Types and control flow**
Use types to express what a value means.

**1.3 Functions and references**
Pass small values by value and inspect larger objects through const references when appropriate.

**1.4 Namespaces and headers**
Headers declare interfaces that other translation units consume; source files provide implementations.

**You will produce:** Reproducible two-file project and lifetime explanation.

## 2 Object-Oriented C++

Model task records with invariants instead of a collection of unrelated fields.

**2.1 Classes and objects**
A class combines state with operations that preserve its rules.

**2.2 Constructors**
A constructor establishes a usable object.

**2.3 Inheritance**
Inheritance models a substitutable relationship, not simply code reuse.

**2.4 Interfaces and polymorphism**
Runtime polymorphism lets a caller use a stable interface while implementations vary.

**You will produce:** Class diagram, invariant tests and interface example.

### Practice is part of every module

Work through the example, record your reasoning, complete the module assignment and compare your work with the review criteria.

DETAILED MODULE CURRICULUM · 3-4

# Apply, integrate and demonstrate.

The module names, lesson sequence and evidence below match the course learning workspace.

## 3 STL and Problem Solving

Store tasks and query them with standard containers and algorithms.

**3.1 Vectors and maps**
A vector owns a sequence; a map associates keys with values.

**3.2 Iterators**
An iterator describes a position in a range.

**3.3 Algorithms**
Standard algorithms separate traversal from the operation being applied.

**3.4 Exceptions**
Exceptions report failures that normal return flow cannot handle locally.

**You will produce:** Container tests and documented error handling.

## 4 Ownership and Capstone

Make resource lifetime predictable and ship a complete console application.

**4.1 Stack and heap**
Automatic local objects are destroyed when their scope ends.

**4.2 Smart pointers**
A unique_ptr expresses exclusive ownership; shared_ptr represents shared ownership with reference counting.

**4.3 RAII**
RAII couples resource acquisition with object lifetime.

**4.4 Console application capstone**
Complete the task utility using values and standard containers.

**You will produce:** Console capstone, ownership map and regression cases.

### Practice is part of every module

Work through the example, record your reasoning, complete the module assignment and compare your work with the review criteria.

PROJECTS AND WORKING METHODS

# A modern C++ console application

Model a practical problem with classes, use standard containers and algorithms, and explain resource ownership in your solution.

**Class and ownership design**

**Container and algorithm examples**

**Tested console project**

## Tools and methods

C++20 compiler | VS Code | Git

Tools are learning choices, not partner endorsements.

## Inside module 1: a worked example

```
#include <iostream>
#include <string>
void greet(const std::string& name) {
  std::cout << "Hello, " << name << "\n";
}
int main() { greet("Learner"); }
```

The function reads the string without mutating it. Its reference is valid for the duration of the call.

## How your work is reviewed

Show the artifact, explain your decisions, test the stated assumptions and identify limitations. Module assignments build toward the course project rather than replacing it with a single example.

DELIVERY, COMPLETION AND NEXT STEPS

# Know what you are signing up for.

### Delivery

Self-paced online · 6-8 weeks

Confirm current schedule, cohort arrangements and enrollment terms with the academy.

### Completion requirements

Complete the required coursework and module assignments, and pass the course assessment. A course completion certificate is available after those requirements are met.

## What the learning workspace includes

### Follow the modules

Expand the curriculum and open the exact lesson you need.

### Apply the concepts

Read explanations, inspect examples and complete module practice.

### Keep your evidence

Use resources, notes, discussion and assignments to support your learning.

### Learning commitments, not outcome guarantees

Completion is not a placement, employment or performance guarantee. The course project is educational evidence; any production use requires appropriate review and validation.

**Start with the curriculum. Continue with the first lesson.**

Review your starting point, download this guide and explore the connected course experience.

Go to the first lesson →

Client-review edition. The static learning workspace saves demonstration progress locally; production enrollment, assessment, communications and certificate issuance are not connected.